About this PDF

Built only from: **NOC:Cloud Computing and Distributed Systems (NPTEL course book, week 5)** — https://nptel.ac.in/courses/106104182

Licence: CC BY-SA 4.0. Accessed 2026-09-26.

Covers 10 of 10 syllabus sub-topics of this unit.

Partly covered in the source:

- Background and Definitions

Consensus and Agreement Algorithms: Problem Definition

Definition

Consensus is the problem of getting all correct processes in a distributed system to agree on a single value.

Explanation

In synchronous message-passing systems, consensus can be achieved even when processes may crash or behave Byzantine. In asynchronous message-passing systems, the FLP impossibility result shows that consensus cannot be solved if even a single process may crash.

Example

When n = 4 and f = 1, the commander is Byzantine and sends 0 to some processes and 1 to others. After exchanging messages, each non-faulty process receives the values 1, 0, 0 and takes the majority value 0, achieving agreement.

Algorithm / Steps

1. **Base case**: If f = 0, the source process sends its value to every other process, and each process uses the received value.
2. **Recursive step**: If f > 0, the source process sends its value to every other process. For each recipient i, let v_i be the value received (or 0 if none). Process i then becomes a new source and invokes the algorithm with f-1.
3. **Majority computation**: After all recursive calls complete, each process computes the majority value among the values received from the other processes. If no majority exists, it defaults to 0.

Key points

- Consensus requires all correct processes to agree on a single value.
- Asynchronous systems cannot solve consensus if a crash may occur.
- The Lamport–Shostak–Pease algorithm solves Byzantine agreement when n >= 3f+1.
- The algorithm uses recursion with f+1 rounds and exponential message complexity.

Overview of Results

Consensus and recovery in distributed computing aim to guarantee that all non-faulty processes agree on a single value, even in the presence of faults and varying system assumptions.

Definition

Consensus is a problem where processes with initial values must decide on a common value satisfying agreement, validity, and termination.

Explanation

Agreement requires all non-faulty processes to agree on the same value; validity ensures that if the source process is non-faulty, the agreed value equals its initial value; termination guarantees that every non-faulty process eventually decides. In the interactive consistency variant, each process has an initial value and the agreed result is an array of values, one per process, with the same array agreed upon by all non-faulty processes. Byzantine agreement is a special case of consensus with a single source; it is solvable under synchronous systems but not in asynchronous ones., Fault models such as crash, omission, and Byzantine influence the feasibility of agreement protocols., Authenticated messages can mitigate Byzantine faults, whereas unauthenticated messages allow tampering. Performance metrics for agreement algorithms include rounds, message traffic, and storage overhead.

Key points

- Consensus requires agreement, validity, and termination among non-faulty processes.
- Byzantine agreement is a consensus variant with a single source and is solvable under synchronous systems but not in asynchronous ones.,
- Fault models such as crash, omission, and Byzantine influence the feasibility of agreement protocols.,
- Authentication of messages can mitigate Byzantine faults, whereas unauthenticated messages allow tampering.
- Performance metrics for agreement algorithms include rounds, message traffic, and storage overhead.

Agreement in a Failure-Free System(Synchronous and Asynchronous)

Definition

Consensus problem is a fundamental problem in distributed systems.

Explanation

In synchronous message-passing systems, consensus can be achieved even when some processes fail, for example under fail-stop or Byzantine fault models. In contrast, in asynchronous message-passing systems, consensus is impossible even if only one process may crash. This impossibility result motivates weaker consensus variants such as terminating reliable broadcast, which requires eventual delivery of messages even when the sender crashes, but does not require termination. The terminating reliable broadcast problem is solvable under crash faults with quadratic complexity. Thus, the solvability of agreement depends on the synchrony assumption. Synchronous systems allow common knowledge of the consensus value to be attained, whereas asynchronous systems cannot guarantee common knowledge.

Key points

- Consensus is a fundamental problem in distributed systems.
- Synchronous message-passing systems can solve consensus even with failures.
- Asynchronous message-passing systems cannot solve consensus even with a single crash fault.
- Terminating reliable broadcast is a weaker consensus variant solvable under crash faults.
- Common knowledge of the consensus value is attainable in synchronous systems but not in asynchronous systems.

Agreement in Synchronous Systems with Failures

Definition

Agreement is the requirement that all correct processes in a distributed system eventually decide on the same value, even when some processes may fail. In a synchronous message-passing system, this agreement must be reached within a bounded number of rounds.

Explanation

In synchronous systems, processes execute in lockstep rounds, and every message sent in a round is received by all correct processes in the next round. This timing guarantees that if a process can send a message, all other correct processes will receive it, allowing them to reach a common decision. The failure models considered include fail-stop, crash, omission, and Byzantine faults, where a faulty process may behave arbitrarily. The system is assumed to have full logical connectivity, reliable channels, and sender identification, simplifying the analysis of agreement protocols.

Because the system is synchronous, the impossibility result that holds for asynchronous systems does not apply, and agreement can be achieved even in the presence of Byzantine faults. However, the algorithm must satisfy validity (the agreed value must be one of the proposed values), integrity (no process can decide on a value that was never proposed), and termination (every correct process eventually decides).

Example

A practical application of agreement in synchronous systems is the atomic commit protocol used in distributed database systems, where all participating nodes must agree to commit or abort a transaction. Another example is fault-tolerant clock synchronization, which requires all correct nodes to agree on a common time value. In systems such as Bitcoin and aircraft flight-control software, Byzantine agreement protocols are employed to tolerate malicious or faulty components.

Key points

- Agreement requires all correct processes to decide on the same value.
- Synchronous execution in rounds guarantees message delivery within a bounded time.
- Validity, integrity, and termination are the core properties of a correct agreement protocol.
- Byzantine faults allow arbitrary misbehavior, yet agreement remains solvable in synchronous systems.
- Applications include atomic commit in distributed databases and fault-tolerant clock synchronization.

Checkpointing and Rollback Recovery: Introduction

Definition

Checkpointing and rollback recovery are schemes that allow a distributed system to recover from failures by restoring the system to a previous error-free state. They are implemented either by checkpointing the state of processes or by logging non-deterministic events.

Explanation

In checkpointing based schemes, each process periodically saves its local state. Coordinated checkpointing ensures that all processes take checkpoints in a coordinated manner to form a consistent global state, thereby avoiding the domino effect. Uncoordinated checkpointing allows each process to take checkpoints independently, which can lead to the domino effect during recovery. Communication-induced checkpointing forces processes to take checkpoints based on piggybacked information in messages, also helping to avoid the domino effect. Log based rollback recovery combines checkpointing with logging of non-deterministic events, exploiting dependencies to record a consistent state.

Example

Suppose process P wants to establish a checkpoint at P3. This records that message q1 was received from Q; to prevent q1 from becoming orphaned, Q must also checkpoint. Thus establishing the checkpoint at P3 forces Q to take a checkpoint to record that Q sent the message.

Algorithm / Steps

1. The initiating process takes a tentative checkpoint and requests all other processes to take tentative checkpoints.

2. Each process takes a tentative checkpoint and records the last message received from each process.
3. All processes send a final decision to the initiator; the initiator decides to make the tentative checkpoints permanent or discard them.
4. In the recovery phase, the initiator sends a message to all processes asking whether they should restart from a previous checkpoint.
5. Each process receives the final decision and either rolls back to a previous checkpoint or continues execution.

Key points

- Checkpointing saves local states; coordinated checkpointing avoids the domino effect.
- Uncoordinated checkpointing has lower runtime overhead but can suffer from the domino effect.
- Communication-induced checkpointing uses piggybacked information to trigger checkpoints.
- Log based rollback recovery combines checkpointing with logging of non-deterministic events.
- The Koo-Toueg coordinated checkpointing algorithm uses tentative and permanent checkpoints to achieve a consistent global state.

Background and Definitions

Partly covered in the source.

Definition

A local checkpoint is a snapshot of the state of a process at a given instance, taken when all processes save their local state at a certain instant of time. Stable storage is a type of storage that survives even after serious faults, and processes store all local checkpoints on it. A global state of a distributed system is a collection of the individual states of all participating processes and the state of the communication channels. A consistent global state is a particular global state that may occur during the failure-free execution of a distributed computation. Coordinated checkpointing is a method where processes coordinate to record checkpoints so that the system-wide state is consistent and the domino effect does not occur. Communication-induced checkpointing forces each process to take checkpoints based on information piggybacked on the application, helping avoid the domino effect. Log-based rollback recovery combines checkpointing with logging of nondeterministic events, exploiting dependencies while taking checkpoints to record a consistent state.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Explanation

Coordinated checkpointing ensures that all recorded checkpoints together form a consistent state, thereby preventing the domino effect. Communication-induced checkpointing uses piggybacked information to trigger checkpoints, which also helps avoid the domino effect during checkpointing. Log-based rollback recovery relies on logging nondeterministic events along with checkpoints, allowing proper exploitation of dependencies and recording of a consistent state.

Key points

- Local checkpoints capture a process's state at a specific instant.
- Stable storage guarantees persistence of checkpoints after faults.
- A global state aggregates all process states and channel states.
- Consistent global states can arise during normal execution without failures.
- Coordinated, communication-induced, and log-based checkpointing schemes all aim to avoid the domino effect by ensuring consistent checkpointing.

Issues in Failure Recovery

Definition

Issues in failure recovery are the problems that arise when a distributed system attempts to restore a consistent state after a fault has occurred. These problems include the need to capture inter-process dependencies, the possibility of cascading rollbacks, and the handling of messages that become inconsistent during recovery.

Explanation

During rollback recovery, each process periodically saves a checkpoint of its state. If a failure occurs, the system must roll back to a previous consistent checkpoint. However, messages that were sent after a checkpoint but received before the failure can become orphan messages, which force non-faulty processes to roll back as well, leading to a domino effect or cascaded rollback. Delayed messages, whose receipt was not recorded because the receiver was down or the message arrived late, also contribute to inconsistencies. Duplicate messages can arise from message logging and replay during recovery. These issues highlight the difficulty of capturing a global consistent state in a distributed environment.

Example

Consider processes P1, Pj, and Pk that have taken checkpoints C1,0, C1,1, Cj,1, Cj,2, Ck,1, Ck,2, and Ck,3. If a failure occurs after P1 has taken checkpoint C1,1, the system must roll back to this checkpoint. The message H sent by Pj to Pk becomes an orphan message because Pj's rollback to Cj,2 does not record the send of H. This orphan message triggers a rollback of Pk, even though Pk was not faulty, illustrating a cascaded rollback.

Algorithm / Steps

1. Detect a failure in the system.
2. Roll back the affected process to its most recent checkpoint.

3. Identify orphan messages whose send events were not recorded in the checkpoint.
4. Trigger rollback of any process that received an orphan message.
5. Repeat steps 3–4 until all affected processes are restored to a consistent state.

Key points

- Rollback recovery requires periodic checkpointing of each process.
- Orphan messages can force non-faulty processes to roll back, causing a domino effect.
- Delayed and duplicate messages further complicate recovery.
- Uncoordinated checkpointing suffers from these issues, whereas coordinated checkpointing can avoid them.
- Log-based recovery schemes (pessimistic, optimistic, casual) differ in how they record determinants to handle nondeterministic events.

Checkpoint-based Recovery

Definition

Checkpoint-based recovery is a backward recovery method that stores snapshots of process states, called checkpoints, to allow a system to roll back to a previous error-free state after a failure.

Explanation

In this scheme, processes coordinate to record checkpoints that form a consistent global state, thereby preventing the domino effect that can occur with uncoordinated checkpoints. Coordinated checkpointing can be blocking or non-blocking, and communication-induced checkpointing forces each process to take checkpoints based on piggybacked information in messages, also avoiding the domino effect. Communication-induced checkpointing includes autonomous and forced checkpoints, and piggybacked data is used to decide whether a forced checkpoint is needed or to advance the global recovery line. Log-based rollback combines checkpointing with logging of nondeterministic events, allowing deterministic replay of events to reconstruct a pre-failure state even if the state was not checkpointed.

Example

An inconsistent checkpoint can arise when a message is sent but not recorded in the sender’s checkpoint, while the receiver’s checkpoint records the message receipt. For instance, the send of message m is not captured in checkpoint c0,x, but the receive of m is captured in checkpoint c1,x, producing an inconsistent global state. If the channel is FIFO, this problem can be avoided by forcing each process to take a checkpoint before receiving the first post-checkpoint message.

Algorithm / Steps

1. A message is sent that tells each process the last message m_pq that process p has received from it, and a tentative checkpoint is recorded by q to prevent message m_pq from being orphaned.
2. Process q is asked to take a tentative checkpoint and record the sending of message m1.
3. If all processes confirm the tentative checkpoints, the tentative checkpoints are converted into permanent checkpoints.
4. Optimizations: if no activity has occurred since the last checkpoint, a process need not be rolled back.

Key points

- Checkpointing stores process states to enable rollback to a valid state.
- Coordinated checkpointing avoids the domino effect by ensuring a consistent global state.
- Communication-induced checkpointing uses piggybacked information to trigger forced checkpoints and avoid the domino effect.
- Log-based rollback complements checkpoints by logging nondeterministic events for deterministic replay.

Coordinated Checkpointing Algorithm

Definition

Koo and Toueg coordinated checkpointing algorithm is a rollback-recovery technique that uses a single initiator to coordinate the creation of a consistent global checkpoint across all processes. It employs two kinds of checkpoints: tentative and permanent.

Explanation

The algorithm assumes FIFO channels and that the network is not partitioned. A single process initiates the checkpointing activity and no process fails during its execution. The checkpointing part has two phases: first, the initiator takes a tentative checkpoint and requests all other processes to take tentative checkpoints; after taking a tentative checkpoint, a process must not send any application messages. In the second phase, all processes receive a final decision from the initiator to either make the tentative checkpoints permanent or discard them. The algorithm guarantees that either all processes take a permanent checkpoint or none do, and that no process sends a message after a permanent checkpoint. Optimizations allow processes that have not changed state since their last checkpoint to skip taking a new checkpoint. The recovery part also has two phases: the initiator asks all processes whether they agree to roll back to a previous checkpoint; after receiving all preferences, it sends a final decision to all processes, which then act accordingly. This ensures that the system can be restored to a consistent state after a failure.

Example

An inconsistent checkpoint can occur when a message `m` is sent by `P0` after the checkpoint request has been sent to `P1`, but `m` reaches `P1` before `P1` receives the checkpoint request. In this case, `c1,x` records the receipt of `m` while `c0,x` does not record that `m` was sent, producing an inconsistent global state. If the channels are FIFO, this problem can be avoided by ensuring that each process takes a checkpoint before receiving the first post-checkpoint message on each channel.

Algorithm / Steps

1. **Phase 1 – Initiation**
   - The initiating process takes a tentative checkpoint.
   - It sends a checkpoint request to all other processes.
   - Each process, upon receiving the request, takes a tentative checkpoint and refrains from sending application messages until the checkpointing activity completes.
   - The initiator collects confirmations from all processes.
   - If all processes agree, the initiator sends a "make permanent" decision; otherwise it sends a "discard" decision. 2.3 Each process, upon receiving the final decision, either converts its tentative checkpoint to a permanent checkpoint or discards it.
2. **Recovery Phase**
   - The initiator sends a rollback request to all processes, asking whether they can restart from a previous checkpoint. 3.2 After receiving all preferences, the initiator sends a final decision to all processes. 3.3 Each process rolls back to the agreed checkpoint or continues execution.

Key points

- The algorithm uses a single initiator and assumes FIFO channels.
- Two checkpoint types: tentative (temporary) and permanent (final).
- All processes either take a permanent checkpoint or none do, preventing orphan messages.
- Recovery is performed by a coordinated rollback to a consistent global state.
- Optimizations allow processes with no state change since the last checkpoint to skip checkpointing or rollback.

Algorithm for Asynchronous Checkpointing and Recovery

The algorithm for asynchronous checkpointing and recovery is a coordinated checkpointing scheme that uses tentative and permanent checkpoints to maintain consistency and avoid the domino effect.

Definition

A coordinated checkpointing algorithm that operates in an asynchronous environment, where a single process initiates the checkpointing activity and all processes take tentative checkpoints before deciding to commit or abort.

Explanation

The algorithm proceeds in two phases. In the first phase the initiating process takes a tentative checkpoint and requests all other processes to take tentative checkpoints; after taking a tentative checkpoint, a process stops sending application messages. In the second phase the initiating process decides whether to commit the tentative checkpoints as permanent or to discard them, and all processes act accordingly. The algorithm guarantees that either all processes take a permanent checkpoint or none do, and that no process sends a message after a permanent checkpoint, ensuring a consistent global state. Optimizations allow processes that have not changed state since the last checkpoint to skip taking a new checkpoint, reducing overhead.

Example

If process P wants to establish a checkpoint at P3, it records that message q1 was received from Q; to prevent orphaned messages, Q must also take a checkpoint. This coordination ensures that the checkpoints at P3 and Q are consistent.

Algorithm / Steps

1. The initiating process takes a tentative checkpoint and sends a request to all other processes to take tentative checkpoints.
2. Each process, upon receiving the request, takes a tentative checkpoint and stops sending application messages.
3. The initiating process decides to commit or abort the tentative checkpoints.
4. If committing, the initiating process sends a commit message to all processes; each process converts its tentative checkpoint to a permanent checkpoint.
5. If aborting, the initiating process sends an abort message; each process discards its tentative checkpoint.
6. For recovery, the initiating process sends a request to all processes to restart to the previous checkpoints; all processes agree to restart or not, and act accordingly.

Key points

- The algorithm uses tentative and permanent checkpoints to avoid inconsistent global states.
- All processes stop sending messages after taking a tentative checkpoint, preventing orphaned messages.
- A single initiator coordinates the checkpointing, ensuring either all or none of the processes commit.
- Optimizations allow skipping checkpoints for processes with no state change, reducing overhead.
- The recovery phase requires agreement from all processes to restart to a previous checkpoint.

Source and licence

This PDF is adapted from **NOC:Cloud Computing and Distributed Systems (NPTEL course book, week 5)** (https://nptel.ac.in/courses/106104182), accessed 2026-09-26.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).